

Dependency Regression Testing - DRT

PI Felix Dobslaw, Mid Sweden University

1 Purpose and aims

Correctness validation through software testing today focuses on eliminating within-project failures that are captured during Regression Testing (RT) over a test suite, a time-consuming but necessary guardrail [9]. External dependencies are commonly mocked or faked when testing for regressions to increase testing efficiency, which allows developers to focus on project-internal logic while reducing test execution time and complexity. However, modern software is still often released after partial RT due to frequent code changes and tight development schedules constraints [16].

Currently, modern software systems heavily rely on external dependencies, including libraries, frameworks, and services. As a result, more faults can escape into production due to dependency regressions, where failures are caused by changes in external dependencies. Dependency regressions are difficult to spot because they can occur without code modifications, thereby hidden in the call stack [3]. Moreover, dependency regressions

are rarely tested because external updates are assumed to be safe. This assumption risks incorrect behavior and exposes a fundamental gap in testing today's complex, interdependent systems. Furthermore, dependency regressions lack a dedicated testing paradigm that explicitly targets correctness risks arising from dependency changes, despite increasing prevalence and impact [20].

This gap is broadening further due to software solutions that combine ML and LLMs with regular coding are growing rapidly. In these systems, external dependencies are more common, which are less transparent, less explainable, and more difficult to manage [8]. Furthermore, ML models and LLMs may change behavior because of retraining, parameter updates, or provider-side modifications, often without clear versioning or explicit change documentation [12,30]. Often it is not possible to re-run complete regression test suites for every dependency update (e.g., LLM model change) due to cost, time constraints, and non-deterministic behavior of these systems [21]. Current methods mostly focus on internal code evolution and provide limited support for detecting failures caused by external updates. Consequently, existing dependency regression testing practices are ill-equipped to ensure correctness in ML- and LLM-intensive systems [8].

In response, this project proposal introduces the novel concept of Dependency Regression Testing (DRT) and highlights the need for formalization and empirical evaluation to reduce failures caused by dependency regressions. The core idea in DRT is to carve out integration points and create unit tests for differential testing. Carving out unit tests for static and deterministic dependencies (lower-left quadrant in Fig. 1) is no new idea [11,24], but with the advent of LLMs holistic solutions that address even complex non-deterministic dependencies that change over time without knowledge (upper-right quadrant of Fig. 1) can now be formalized and dependency regression tests automatically extracted/created. The four-year DRT project consists of four main activities (A1-A4) that all contribute to addressing the overarching research question

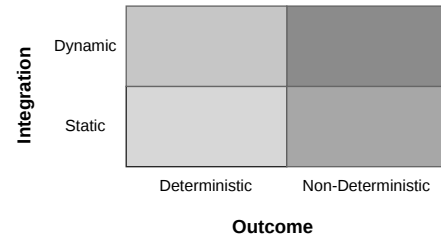


Figure 1: DRT covers multiple regressions.

RQ: How to effectively and efficiently detect and resolve dependency regressions?

While RT for reasons of efficiency mocks out as much dependency code as possible from a project (see blue area in Fig. 2), DRT instead carves out minimal dependency tests (DT) around the same integration points to focus on fast dependency correctness validation with minimal environmental setup. Thus, DRT can be triggered whenever a dependency change is anticipated or enforced, without the need for RT, as illustrated in Fig. 3.

With DRT, dependency updates can be handled through a different and substantially faster validation path. Similar to RT, DRT validation can be limited to those dependencies affected [9], but now much more fine-grained and through carved-out unit tests, making use of replayed input/output pairs for previously passing tests acting as oracles. Similar to how meta-data about test outcomes gets collected for RT today, DTs are extracted from instrumented previous runs of the test suite, where the integration points inputs and outputs are recorded and replayed for correctness validation. DRT is thus thought of as a complement to RT, and one that makes no assumptions on the availability of source code to dependencies. Today there is only limited and scattered related work on dependency testing. Existing research primarily focuses on RT within project boundaries, whereas dependency related failures are typically treated only indirectly through integration testing or system testing, and there is no unified conceptual framework that dedicatedly addresses dependency regressions as a distinct testing problem. Therefore, a unifying formalization and evaluation effort is called for to establish DRT as a method and tool. DRT test cases are minimal, with the consequence that in case of failure, causes are spotted immediately and resolved quickly. Today, developers need to run slow RT and debug more complex tests, such as integration tests, with causes hidden deep in the stack trace. In contrast, DRT isolates dependency behaviour at integration points, making the source of failures significantly easier to identify and diagnose. This improves developer productivity and reduces the time required to

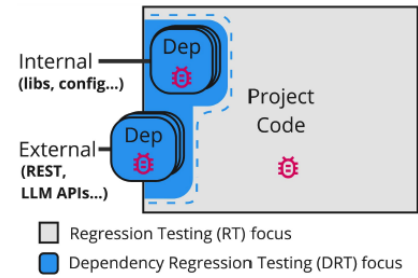


Figure 2: Carved out tests need less environmental setup.

Beyond complementing RT, DRT as a building block can be leveraged to early and atomically signal regressions to dependency maintainers with potentially curating impacts on the entire code ecosystem.

In practice, automatically generated dependency tests could be shared with maintainers through pull requests or issue reports, providing concrete evidence of behavioural regressions. Such feedback can help maintainers detect and resolve issues earlier, benefiting the wider developer community that depends on the same libraries or services. For instance, DRT can be triggered through tools like Dependabot which helps developers stay up-to-date with dependency versions, and can itself propagate pull-requests to dependency maintainers. Reportedly about half the dependency updates through Dependabot-enforced security pull requests are accepted [23], and adding DRT into continuous integration and delivery (CI/CD) pipelines drives adoption as RT can be avoided. Dependency tests directly cut to the functional implications, and if not affected, vulnerability fixes can be applied faster and with minimal human intervention. CI/CD pipelines can trigger DRT, even improving overall security on a project and system level. As a consequence, the failure-finding capabilities for highly used libraries and services get strengthened based on relevant, actual use cases.

The PI has worked on formalizing/solving computer science problems for the past 15 years, lately doing fundamental research on automatic detection of boundary behaviour in software, as well as regression testing research with industry [7]. DRT is a natural extension of previous work on practically relevant foundational research. Following is a brief summary of the aims of

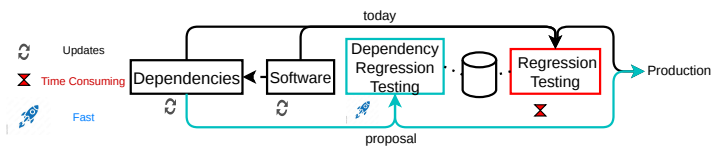


Figure 3: DRT runs real-world tests and reduces runtime.

the four activities that make up this project with their expected outcome:

A1 Defines and establishes Dependency Regression Testing (DRT) as a testing method in the software testing literature with its fundamental elements, in response to the identified gap.

A2 Formalizes and evaluates deterministic DRT, including the investigation of dependency test carving based on instrumented test replays for deterministic functions.

A3 Formalizes and evaluates non-deterministic DRT, expanding and generalizing A2 by non-deterministic functions considering syntactical and semantic result variability/correctness.

A4 Produces an optimization and prioritization plan and methodology for DRT to reduce redundancy and make sure faults are captured early. This activity includes the handling of the growing body of historical replay data.

2 State-of-the-art

Regression testing and its limitations. RT is crucial for software quality but faces challenges like redundancy, repetition, and missing functionalities due to its high complexity [6]. RT re-executes the whole test suite to detect faults after changes, but time and cost constraints limit exhaustive testing for large systems [5]. Furthermore, RT scales poorly—quadratic in both the number of test cases and functions—even when limited to within-project failures, as is the case today. While used extensively in industry, evaluations of RT for large real-world systems are scarce, with one recent exception of a state-of-the-art method, MC-TOA [7]. Finally, while AI techniques have since long been widely applied to RT improvements, the literature centers around in-project regressions, and exclusively for traditional software [7, 32].

Non-determinism and flaky behavior. Test outcomes are not always deterministic, i.e., pass or fail, because of flakiness problems, where outcomes vary due to testing and production environment differences (e.g, hardware differences), which is a significant issue in RT. For example, Wing Lam et al., identified 245 flaky tests over 55 Java projects and found that 75% become flaky when first added [18]. G. Pinto et al., executed 6.4 million test runs over 64,000 tests and achieved 95% F-measure in predicting flakiness [27]. Though flaky test research mainly considers single-run failures [26], repeated-run inconsistencies caused by non-idempotent dependency calls are also common [31], and current RT does not adequately address consequences.

Dependency, failures, and testing approaches. Software testing fails because dependencies are a significant and measurable problem. A study in the Robot Operating System found that 15% of all reported bugs are dependency bugs [13]. Another study looked into eight years of development among 154 developers and found that all types of dependencies increase fault proneness. Among them, logical dependencies explain the most variance in failure rates, followed by workflow, and syntactic dependencies [4]. Furthermore, a systematic literature review provided a catalogue of dependency-related challenges in open-source software package registries [22]. Furthermore, Frank Reyes et al., created a benchmark of 571 reproducible breaking dependency updates across 153 Java projects, which shows the prevalence of dependency-induced failures in software systems [28].

In contrast, a framework for regression test case selection using object dependency graph is proposed as a change identifier and to identify the test cases worthwhile to be rerun in object-oriented software [19]. Another research proposed PRADET, a data-driven method for detecting problematic test dependencies [14], which can scale to large projects with thousands of tests, and has uncovered 27 previously unknown dependencies. Furthermore, automatically linking test cases to changes outside the actual code in a project (e.g., from configuration files) has successfully been demonstrated [25] and been coined Unified Regression Testing [30]. These links can directly be read from dependency management systems (e.g., Gradle for Java), or API calls through specific communication libraries (e.g., RESTEasy for Java) that can be extracted directly without the need for historical test outcomes. Bloated, i.e., unused dependencies in build paths [29], do not pose a problem for DRT, as they are not executed, and

thus need not be carved out. For API dependencies such as through REST, Differential Testing that compares different versions of the same API on the same input has been applied [15]. For DRT and similar to RT, historical data is collected and used for structural replay inspired by Differential Testing. Because API contract evolution (such as for OpenAPI specifications) technically happens outside the project boundaries, it has traditionally not been considered. Overall, no unified dependency regression formalisation exists, calling for a need to unify and expand the existing research into a common methodology that generalizes to all types of project dependencies.

Responsibility of dependency maintainers. One could argue that dependency hygiene is not the responsibility of the user (dependee) but that of popular libraries and service (dependency) maintainers. Still, even for open-source code of the highest sensitivity and impact, such as OpenSSL vulnerabilities (such as Heartbleed) have been in the open and exploited for years before they were found and fixed. Also, current RT emphasizes the maintainer's responsibility to create appropriate test cases without actual use cases, unlike DRT.

Hybrid systems, LLMs, and semantic non-determinism. When considering the fast-growing complexity of software due to hybridization with LLMs, we face the problem of semantic correctness, i.e., multiple answers being equally or similarly correct. Correctness thereby becomes fuzzier (how correct does an answer need to be?) and with ML/LLM integration, regressions can slip through in the background, such as through model changes [21]. These facts require a new methodology for correctness validation, and no commonly agreed terminology exists, and very little method today to build upon [8]. We argued earlier for a duality in testing traditional and ML software [9], with a commonality of non-determinism, rarely assumed in the literature.

LLMs though, bring in another level of non-determinism, as they even if configured (e.g., temp=0) are not guaranteed to be deterministic [2]. ML models give the same result for a query, the non-determinism there stems from the lack of representative training data [9]. Confidence in ML or similarity among LLM responses can be assessed in probabilities, such as through the cosine similarity measure. In light of the semantic non-determinism, DRT becomes even more pertinent and needs to consider several variation points that the testing community only started to define and understand better [8]. The fact that the correctness of LLM calls can inherently not be assessed by single execution of a model for a prompt. Also, recent approaches to lump together accuracy statistics for varying kinds of requests to the same LLM dependency do not produce feedback that helps pinpoint the weaknesses of the LLM [21]. The work on DRT will define a structure to integrate the future contributions in this area for non-deterministic correctness assessment through Aggregated Oracles [8].

3 Significance and scientific novelty

The DRT project has significant implications within software engineering research, extending the software-testing state-of-the-art conceptually and empirically to address the growing methodological gap in quality assurance for complex software systems. It does that by establishing novel complementing principles for effective and efficient RT that

- go beyond the limiting deterministic testing assumption in the regression testing literature.
- go beyond the prevalent but limiting assumption that coverage assessment is limited to within-project functionality. Instead of expanding on poorly scaling coverage matrice calculations, DRT here introduces a scalable complement.
- span highly problematic phenomena not previously covered, such as flakiness and semantic correctness variation in dependencies of hybrid and non-deterministic systems including LLMs.

The project results will be relevant for software engineers/researchers as DRT conceptualizes and generalizes a growing problem from scattered literature around dependency testing, to be able to communicate, use, refine, and challenge or extend dependency testing.

The project results will be relevant for researchers and practitioners applying Generative AI as they contribute to the LLM correctness validation research, i.e. Aggregated Oracles. The results will also be relevant for engineering students in both fields as they contribute with a widened theory on RT to be learned in university curricula.

The results will also be relevant to the open-source community. Negative DT outcomes can be communicated to dependency project owners, e.g. through pull requests, with likely relevant and not yet included test cases. This in return should have failure-finding capabilities for popular libraries go up with minimal effort by the maintainers. The results are relevant for the entire software industry, as DRT can even be utilized as a kind of fast circuit breaker to ensure that software updates after a change only if reliable.

Ultimately, the project results are relevant to the sustainability of the planet. DRT has multiple application areas not limited to externally enforced regressions, but in particular for LLMs - that can act as drop-in replacements for one another - to the strategical change of models, e.g. to reduce costs or environmental impact. DRT will offer fundamental technology to identify good enough LLMs that fulfil a specification. A fast DRT will also lead to solutions for order-dependency issues since DRT makes replay sequences explicit, which in return simplifies the identification of issues even for stateful services, and inconsistent or flaky behaviour in dependencies.

4 Preliminary and previous results

PI Felix Dobsław (FD) has a history of formalizing and solving fundamental and industrial research problems using a broad spectrum of heuristic and optimal methods, recently, particularly in the domain of software testing. The contexts often involved complex, non-differentiable, highly constrained optimisation problems with nondeterministic components, including harsh time limits and needing a good understanding of information theory concepts. The understanding of the fundamental need for DRT grew from his work and contributions in previously funded VR research projects BaseIT and AQUAS (both with Robert Feldt as PI) in test case automation [1, 7] and RT research for large-scale industry systems [9]. The latter led to the understanding that a fundamentally different approach to the coverage of correctness due to external dependencies is needed to make RT scalable and meet the quality requirements in systems with thousands of tests that run for weeks if executed fully. It resulted in the **MC-TOA** framework which uses integer/linear programming for RT evaluated for effectiveness and efficiency in practice which is used in CI/CD pipelines at Huawei today.

During the PhD, FD created **SchedEx**, a scheduling technique giving arrival guarantees for use in mission-critical applications [10] which is used in production worldwide today for heavily non-deterministic systems. Recently, his work has included the limitations of current methods and tools for non-determinism in LLM testing, in particular, semantic non-determinism which needs more attention [8]. The work highlights that research on the Oracle problem for LLMs has so far been very limited and the formalization of Aggregated Oracles that tests run multiple times to aggregate over the solution, needs development and empirical evaluation which the DRT project will contribute to.

Our recent findings on the correlation between architectural smells and quality attributes such as testability in software projects [17] further heighten the awareness of the need for smart use of dependencies that focus on integration points rather than increasing the internal complexity and nature of the code [8].

The preliminary conceptualization of DRT in this proposal is a first start. DTs, i.e. unit tests at integration points, require minimal infrastructure such as hardware or virtual environments - those carved-out parts are only needed for regular RT for changes to the project code. FD has heavily worked with industry and development of frameworks in different domains. Within software testing, he was the main author of AutoBVA, the first Automated Boundary Value Analysis framework for the automated extraction of boundary candidates [7], and co-supervises

the continuation work [1].

5 Project description

5.1 Theory and method

Research contributions will be developed in cycles of literature review, idea conception, prototyping, experimentation, and evaluation. Details on the scientific approach, as well as evaluation, will be discussed in the following sections.

5.2 Time plan and implementation

The four activities (**A1-A4**) illustrated in Fig.4, all address specific research questions and an expected outcome. **A1** spans the entire project starting with a unification of the state-of-the-art to then continuously refine and generalize DRT. **A2** and **A3** then contribute to the project by incrementally formalizing and evaluating important parts of the DRT method. These activities will provide the theoretical and empirical foundations needed to demonstrate the feasibility and effectiveness of DRT in practice.

In **A2**, we use the instrumented replayed tests and code to carve out valid and isolated unit tests for libraries and API calls. Beyond instrumentation, **A2**, investigates existing projects leveraging existing instrumentation tools, to understand what categories of integration points need to be differentiated, also to better understand their practical challenges, while investigating formalisation differences. **A3** disseminates literature and integrates non-deterministic oracle types into the project for traditional code, ML models, and LLMs; also looks into existing projects to identify LLM integrations and extracting DTs. **A4** then investigates patterns in the historical records from RT used to replay in DRT and proposes techniques to optimize the process, inspired by the MC-TOA framework [9]. It here follows a detailed description of the activities, the research questions they address, and the expected outcome:

In **A2**, we use the instrumented replayed tests and code to carve out valid and isolated unit tests for libraries and API calls. Beyond instrumentation, **A2**, investigates existing projects leveraging existing instrumentation tools, to understand what categories of integration points need to be differentiated, also to better understand their practical challenges, while investigating formalisation differences. **A3** disseminates literature and integrates non-deterministic oracle types into the project for traditional code, ML models, and LLMs; also looks into existing projects to identify LLM integrations and extracting DTs. **A4** then investigates patterns in the historical records from RT used to replay in DRT and proposes techniques to optimize the process, inspired by the MC-TOA framework [9]. It here follows a detailed description of the activities, the research questions they address, and the expected outcome:

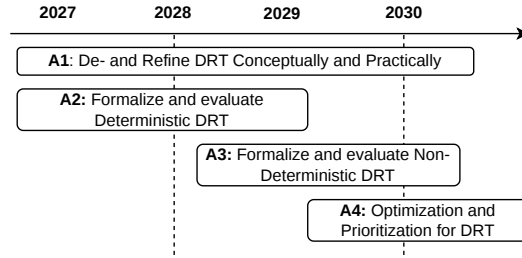


Figure 4: Timeline for DRT project.

A1: De- and Refine Dependency Regression Testing (DRT) conceptually

The literature is structurally investigated and DRT is conceptualized as a complementary correctness validation method and tool for the software engineering community. Throughout the project, and through incremental empirical studies, an open-source tool will be created to refine the understanding and shape the method. One important question to be analyzed is the relation between complementing DRT and RT, and in what situations either is mandatory/optional.

RQ1: What is DRT and how does it complement traditional RT for correctness validation?

Expected Results: A unified view of the DRT relevant literature, including a systematic taxonomy of dependency regression problems and testing strategies, with a clear conceptualization of what constitutes DRT, its building blocks, and specific challenges. In addition, the project will deliver prototype tooling, empirical evidence from open-source systems, and methodological guidelines for integrating DRT into modern CI/CD pipelines.

A2: Formalize Deterministic Dependency Regression Testing (DRT)

Traditional software integrations have dominantly deterministic input/output relationships. To formalize deterministic DRT, types of determinism will be deduced from a broader mining study over open-source projects for human-identified and labelled dependency integration points. Two tasks follow; automating 1) the finding of integration points, and 2) carving out dependency tests, i.e. compilable unit-test. Finding integration points will be done through static analysis and the extraction of dependency tests using LLMs. Through instrumentation, the integration

point's input/output pairs will be recorded during RT and stored in a database, to later be replayed in the respective dependency tests during DRT. Initially, integration points are assumed to be stateless and independent. This will lead to failures in replay for stateful calls that require the consideration of sequences in the replay. An algorithm for replay sequences is therefore designed thereafter.

RQ2: How to formalize and implement deterministic DRT?

RQ 2.1: What are the different types of deterministic dependency integration points?

RQ 2.2: How to identify and carve out a project's code integration points to a dependency?

RQ 2.3: How to extract executable dependency tests from the integration points?

RQ 2.4:How to deal with stateful dependencies in DRT?

Expected Results: A formalization of deterministic DRT and implementation in a tool that is empirically evaluated for open-source projects.

A3: Formalize Non-Deterministic Dependency Regression Testing (DRT)

Currently, hybridizations of traditional, ML, and LLM software is exploding. For instance, tool integration for LLMs - where LLMs get access to command line tools and their specification are now standard use cases to augment the developer where needed/desired from coding commands (how) to semantical instructions for the LLM (what). The LLM then in return handles the coding. Testing strategies for correctness validation must adapt to this new reality. **A3** focuses on generalising the ongoing DRT formalism to non-deterministic integration points. LLMs can be expected to return semantically similar, but structurally/syntactically different outputs for the same input and DRT must handle more advanced, so-called Aggregated Oracles [8].

Constraint-respecting and coverage-optimal regression plans reduce the number of necessary tests by roughly an order of magnitude in traditional software [9]. However, assessing confidence in the face of non-determinism requires repetition and statistical testing methods. **A3** evaluates, disseminates and aggregates Oracle models coming up during the upcoming 1.5-2 years, to understand their incorporation into DRT. Many repetitions for LLM calls can be slow, which is why efficient and confident replay is required to address semantic Differential Testing during DRT. Built-in quality measures such as cosine similarity, while not perfect, offer a baseline to be tested against. Another level of complexity addressed in the formalization will be the fact that multiple LLMs can be used simultaneously in different integration points. The distinction between non-determinism due to non-availability, flakiness, ML-model confidence, or state issues on one side, and semantic misinterpretations or capability limits on the other side, will be further sharpened through this activity, which may further lead to contributions to the Aggregated Oracle literature.

RQ3: How to formalize and implement non-deterministic DRT?

RQ 3.1: What are the different types of non-deterministic dependency integration points?

RQ 3.2: How to extract dependency tests from non-deterministic integration points?

RQ 3.3: How to deal with flaky dependencies in DRT?

Expected Results: A generalization of DRT for non-determinism, including an implementation in a tool, empirically evaluated for open-source projects.

A4: Optimization and Prioritization of Dependency Regression Testing (DRT)

This activity focuses on efficient DRT executions, given that replays from multiple RTs and DRTs accumulate over time. It will further emulate the development of DRT using and replaying the history of existing open-source projects from A2, to learn redundancy patterns within the obtained test data. Guided by the findings, A4 proposes a suitable solution/method considering both heuristic and convex solutions and multi/many-objective approaches. Aspects such as how constrained the formal problem presentation turns out will heavily affect the choice of method. For example, evolutionary algorithms are good for non-differentiable problem formulations, but convex optimizers suit well for highly constrained situations. Also, hybridization is an option, such as for MC-TOA [9]. Another dimension to be taken into account is including

execution metrics such as time or memory use in both the optimization and the DRT formalism as a whole.

RQ4: What strategies can be applied to optimize DRT over time considering a growing replay base?

RQ 4.1: How can we formulate an optimization problem that minimizes the number of historical tests required to validate correctness?

RQ 4.2: How to generalize DRT to multiple objectives (e.g., runtime or memory footprint)?

Expected Results: A formalization and appropriate method for the problem of optimizing and prioritizing dependency tests. An empirical evaluation of the based on relevant open-source projects compared to an exhaustive baseline.

Openness is a driving force for the project. Publications will be published in open-access venues of the highest scientific quality. Also, all algorithms, software and results produced will be made freely and transparently available as open-source through GitHub and Zenodo - also for reasons of reproducibility (no patents out of principle). Further, the project will be conducted and formed in direct and continuous collaboration with world-leading testing experts in the PIs network. Therefore, 100 TSEK each year are allocated for knowledge dissemination (travel/visits), as well as conference participation (such as ICSE and FSE).

DRT is in its use patterns similar to RT which has been adopted in industry for decades. So while human use is not within the scope of the DRT project, it is conceptually understood to act on a very well-understood interface, with vast implications for efficiency and sustainability. Also, while security is of great importance, DRT focuses on functional software correctness, a subject that is often reduced to coverage or known bug-finding capabilities in the literature, for which both have their limitations. The main project risk is that developing reliable technical solutions within A2 is harder than anticipated. The project's success is not conditioned on that fact, as the tooling is only one contribution that can be improved over time and even beyond the DRT project (RT is not a solved problem either).

5.3 Project organisation

The four-year project will be carried out at Mid Sweden University in the Software Engineering and Education (SEE) group led by the PI. It has four main activities, illustrated in Fig. 4 and the work is carried out by the PI (32% FTE) and co-PI Khan Mohammad Habibullah (80% FTE, 28 months), who works as a Junior lecturer and starts his Postdoc in May 2026.

6 Equipment

Mid Sweden University provides office space and standard equipment.

7 Need for Research Infrastructure

Potentially required computational resources will be arranged through the Swedish NAISS Infrastructure for Computing¹, at no charge to VR-funded projects. Experiments for DRT are not conditioned to special hardware, and negligible use of external APIs, including LLM use.

8 International and national collaboration

FD collaborates with world-leading software engineering researchers, particularly within software testing. Three close partners are mentioned below, and the budget even contains travels for on-site exchanges with partners for necessary co-production that strengthens both this project and the overall dialogue and exchange within the research community.

¹www.niss.se

Name	Affiliation	Area(s)
Prof. Robert Feldt	Chalmers, Gothenburg	RT, LLM, LLM/SW Testing
Prof. Shin Yoo	KAIST, Seoul, Republic of Korea	RT, LLM/SW Testing
Prof. Paolo Tonella	USI, Lugano, Switzerland	ML, ML/SW Testing

Table 1: Potential Collaborators

9 Other applications and grants

FD acted as co-PI in a VR-funded project name AQUAS which ended in 2025. He also worked as a Postdoc in another VR-funded project; BaseIT. In BaseIT and AQUAS FD with the PI developed fundamental notions of diversity in software testing, and then exploited this for automated testing of boundaries in traditional and AI/ML software. FD also co-supervises a fully-funded WASP PhD student in boundary analysis at Chalmers University of Technology (project Bound-Minder, 2024-2028). Project DRT attempts to solve another essential Software engineering challenge that benefits from FD’s background in multi-objective and convex problem solving, but there is no overlap in the research questions of the projects. FD is funded by the GovTech Challenges Research School at Mid Sweden University (2024-2026), where his work focuses on the impact of data and information quality on decision-making processes. Finally, he leads a knowledge foundation (KKS)-funded project on architecture testing through GenAI together with Volvo Cars AB (2025-2027).

References

- [1] Akbarova, S., Dobslaw, F., Gomes de Oliveira Neto, F., Feldt, R.: Setbve: Quality-diversity driven exploration of software boundary behaviors. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2026, accepted for publication)
- [2] Atil, B., Chittams, A., Fu, L., Ture, F., Xu, L., Baldwin, B.: Llm stability: A detailed analysis with some surprises. *arXiv e-prints* pp. arXiv-2408 (2024)
- [3] Böhme, M., Oliveira, B.C.d.S., Roychoudhury, A.: Regression tests to expose change interaction errors. In: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. pp. 334–344 (2013)
- [4] Cataldo, M., Mockus, A., Roberts, J.A., Herbsleb, J.D.: Software dependencies, work dependencies, and their impact on failures. *IEEE Transactions on Software Engineering* **35**(6), 864–878 (2009)
- [5] Dahiya, O., Solanki, K.: Comprehensive cognizance of regression test case prioritization techniques. *International journal of emerging trends in engineering research* **7**(11), 638–646 (2019)
- [6] Dalal, S., Solanki, K., et al.: Challenges of regression testing: A pragmatic perspective. *International Journal of Advanced Research in Computer Science* **9**(1) (2018)
- [7] Dobslaw, F., Feldt, R., de Oliveira Neto, F.G.: Automated black-box boundary value detection. *PeerJ Computer Science* **9**, e1625 (2023)
- [8] Dobslaw, F., Feldt, R., Yoon, J., Yoo, S.: Challenges in testing large language model based software: A faceted taxonomy. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2026, accepted for publication)
- [9] Dobslaw, F., Wan, R., Hao, Y.: Generic and industrial scale many-criteria regression test selection. *Journal of Systems and Software* **205**, 111802 (2023)
- [10] Dobslaw, F., Zhang, T., Gidlund, M.: End-to-end reliability-aware scheduling for wireless sensor networks. *IEEE Transactions on Industrial Informatics* **12**(2), 758–767 (2014)
- [11] Elbaum, S., Chin, H.N., Dwyer, M.B., Dokulil, J.: Carving differential unit test cases from system test cases. In: *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*. pp. 253–264 (2006)
- [12] Eyuboglu, S., Goel, K., Desai, A., Chen, L., Monfort, M., Ré, C., Zou, J.: Model change-lists: Characterizing updates to ml models. In: *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. pp. 2432–2453 (2024)

- [13] Fischer-Nielsen, A., Fu, Z., Su, T., Wasowski, A.: The forgotten case of the dependency bugs: on the example of the robot operating system. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice. pp. 21–30 (2020)
- [14] Gambi, A., Bell, J., Zeller, A.: Practical test dependency detection. In: 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST). pp. 1–11 (2018). <https://doi.org/10.1109/ICST.2018.00011>
- [15] Godefroid, P., Lehmann, D., Polishchuk, M.: Differential regression testing for rest apis. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 312–323 (2020)
- [16] Harrold, M.J., Orso, A.: Retesting software during development and maintenance. In: 2008 Frontiers of Software Maintenance. pp. 99–108. IEEE (2008)
- [17] Jolak, R., Karlsson, S., Dobslaw, F.: An empirical investigation of the impact of architectural smells on software maintainability. *Journal of Systems and Software* **225**, 112382 (2025)
- [18] Lam, W., Winter, S., Wei, A., Xie, T., Marinov, D., Bell, J.: A large-scale longitudinal study of flaky tests. *Proceedings of the ACM on Programming Languages* **4**(OOPSLA), 1–29 (2020)
- [19] Larprattanakul, A., Suwannasart, T.: An approach for regression test case selection using object dependency graph. In: 2013 5th International Conference on Intelligent Networking and Collaborative Systems. pp. 617–621 (2013). <https://doi.org/10.1109/INCoS.2013.115>
- [20] Liu, C., Chen, S., Fan, L., Chen, B., Liu, Y., Peng, X.: Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In: Proceedings of the 44th International Conference on Software Engineering. pp. 672–684 (2022)
- [21] Ma, W., Yang, C., Kästner, C.: (why) is my prompt getting worse? rethinking regression testing for evolving llm apis. In: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI. pp. 166–171 (2024)
- [22] Mens, T., Decan, A.: An overview and catalogue of dependency challenges in open source software package registries. *arXiv preprint arXiv:2409.18884* (2024)
- [23] Mohayjeji, H., Agaronian, A., Constantinou, E., Zannone, N., Serebrenik, A.: Securing dependencies: A comprehensive study of dependabot’s impact on vulnerability mitigation. *Empirical Software Engineering* **30**(3), 89 (2025)
- [24] Monce, G., Degueule, T., Falleri, J.R., Robbes, R.: Client-library compatibility testing with api interaction snapshots. In: 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME). pp. 791–796. IEEE (2025)
- [25] Nanda, A., Mani, S., Sinha, S., Harrold, M.J., Orso, A.: Regression testing in the presence of non-code changes. In: 2011 Fourth IEEE International Conference on Software Testing, Verification and Validation. pp. 21–30. IEEE (2011)
- [26] Parry, O., Kapfhammer, G.M., Hilton, M., McMinn, P.: A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology (TOSEM)* **31**(1), 1–74 (2021)
- [27] Pinto, G., Miranda, B., Dissanayake, S., d’Amorim, M., Treude, C., Bertolino, A.: What is the vocabulary of flaky tests? In: Proceedings of the 17th International Conference on Mining Software Repositories. pp. 492–502 (2020)
- [28] Reyes, F., Gamage, Y., Skoglund, G., Baudry, B., Monperrus, M.: Bump: A benchmark of reproducible breaking dependency updates. In: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). pp. 159–170. IEEE (2024)
- [29] Soto-Valero, C., Harrand, N., Monperrus, M., Baudry, B.: A comprehensive study of bloated dependencies in the maven ecosystem. *Empirical Software Engineering* **26**(3), 45 (2021)
- [30] Wang, Z., Machida, F.: Maintaining performance of a machine learning system against imperfect retraining. In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC). pp. 1782–1787. IEEE (2024)
- [31] Wei, A., Yi, P., Li, Z., Xie, T., Marinov, D., Lam, W.: Preempting flaky tests via non-idempotent-outcome tests. In: Proceedings of the 44th International Conference on Software Engineering. pp. 1730–1742 (2022)
- [32] Yoo, S., Harman, M.: Regression testing minimization, selection and prioritization: a survey. *Software testing, verification and reliability* **22**(2), 67–120 (2012)